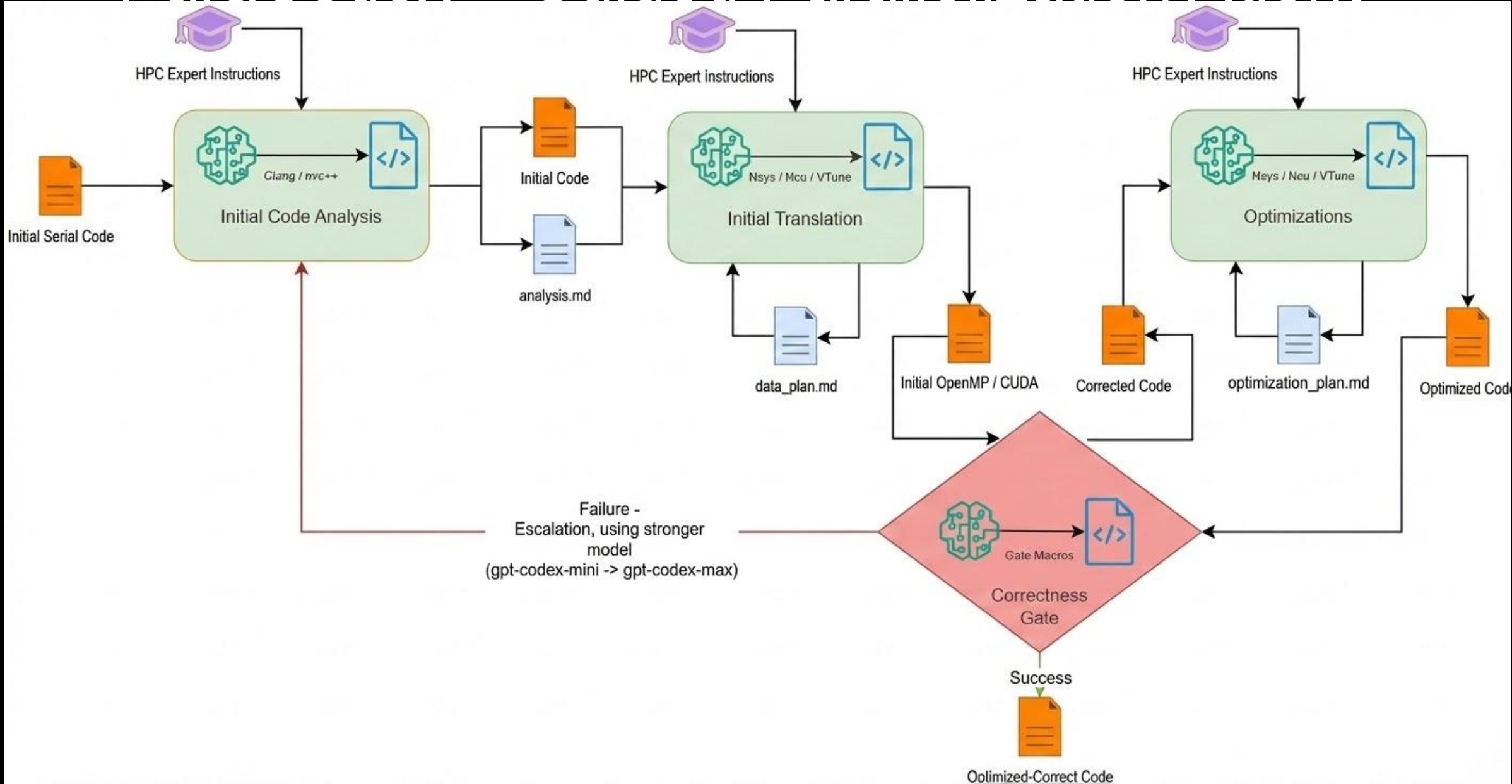


ParaCodex Results

ParaCodex – Guided Parallelization Agent



Benchmarks

HECBench

- Q: Can ParaCodex reliably translate and parallelize small, isolated HPC kernels with clear loop structures?
- Used to validate baseline correctness, loop-parallelization quality, and GPU offload stability.

Rodinia

- Q: Can ParaCodex handle real, irregular, application-level GPU workloads with complex memory patterns and control divergence?
- Tests scalability of the agentic pipeline, robustness on heterogeneous domains, and its ability to match real-world GPU programming idioms.

NAS Parallel Benchmarks (NPB)

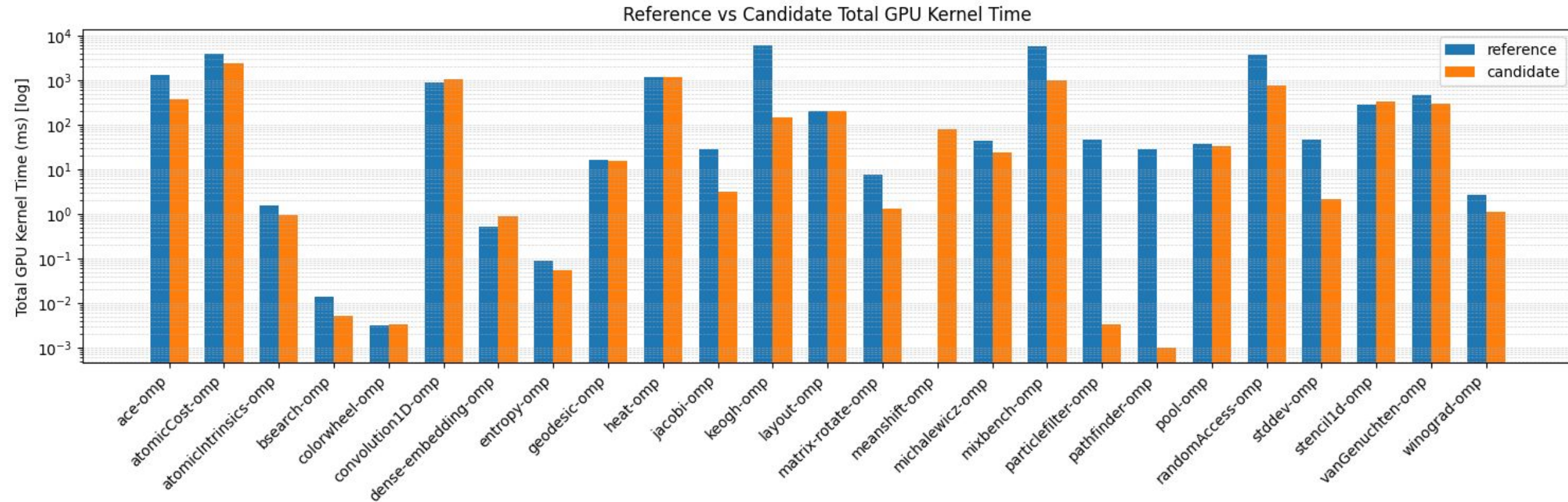
- Q: Can ParaCodex handle a performance-aware parallel code for classic HPC motifs that stress communication, decomposition, and nested parallel loops?
- Evaluates whether the system can achieve high-quality parallelization and offload on canonical scientific kernels used for cluster-level performance studies.

Benchmarks

ParEval

- Q:Can Codex translate between parallel api's?
- Tested *Codex* on the Pareval kernels to measure the capabilities of codex on translating code between parallel api's.

RQ1: Can ParaCodex create correct & optimized parallel code ?
ANS1: Yes, on 76% of the Hecbench codes

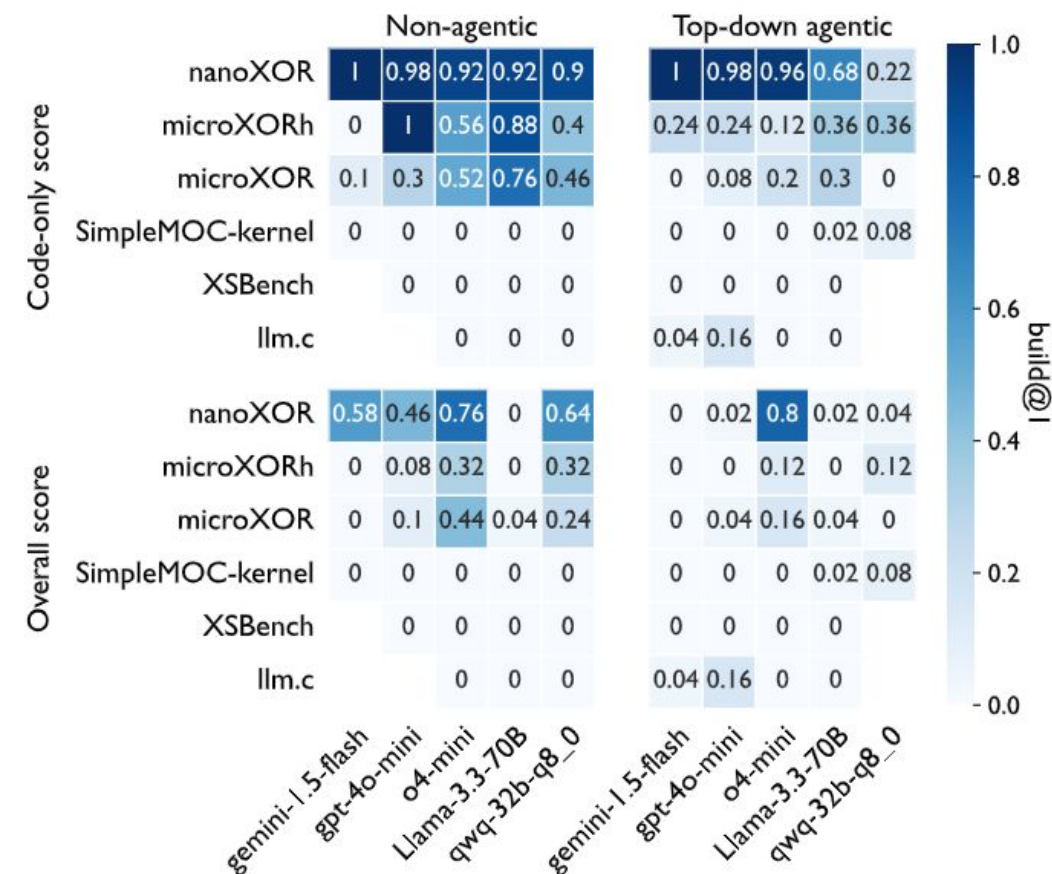


RQ1: Can ParaCodex create correct & optimized parallel code ?
 ANS1: Yes, often by a lot

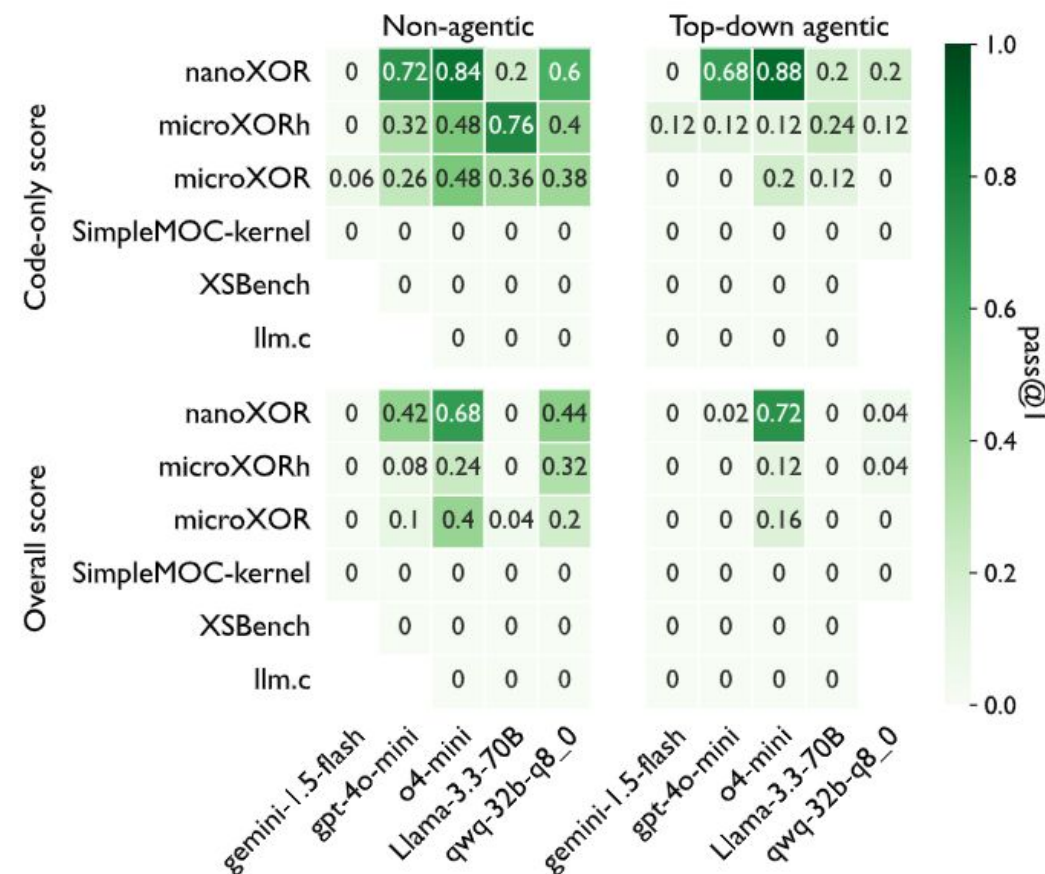
kernel-name	Reference (ms)	candidate-ParaCodex (ms)	Speedup	1-if speedup 0-if not
ace-omp	1271.5508	363.7893	3.4953	1
atomicCost-omp	3840.8302	2336.4017	1.6439	1
atomicIntrinsics-omp	1.5507	0.9494	1.6334	1
bsearch-omp	0.0138	0.0053	2.6060	1
colorwheel-omp	0.0032	0.0033	0.9739	0
convolution1D-omp	907.2729	1041.9738	0.8707	0
dense-embedding-omp	0.5173	0.8741	0.5918	0
entropy-omp	0.0895	0.0543	1.6474	1
geodesic-omp	16.6456	15.6171	1.0659	1
heat-omp	1194.7852	1198.7119	0.9967	0
jacobi-omp	27.4342	3.0859	8.8901	1
keogh-omp	6186.7967	142.5745	43.3934	1
layout-omp	206.4583	204.9066	1.0076	1
matrix-rotate-omp	7.6142	1.2917	5.8945	1
meanshift-omp	nan	80.3619	nan	0
micchalewicz-omp	43.3745	24.4771	1.7720	1
mixbench-omp	5885.8227	1010.8261	5.8228	1
particlefilter-omp	46.2105	0.0033	14065.6898	1
pathfinder-omp	28.3866	0.0010	28625.1526	1
pool-omp	37.7783	33.8897	1.1147	1
randomAccess-omp	3631.2508	744.4224	4.8779	1
stddev-omp	45.7333	2.0915	21.8666	1
stencil1d-omp	289.0810	336.8094	0.8583	0
vanGenuchten-omp	459.2897	302.2992	1.5193	1
winograd-omp	2.6092	1.0968	2.3790	1

RQ2: Can Codex Translate between Parallel API's ?

ANS2: CUDA TO OMP



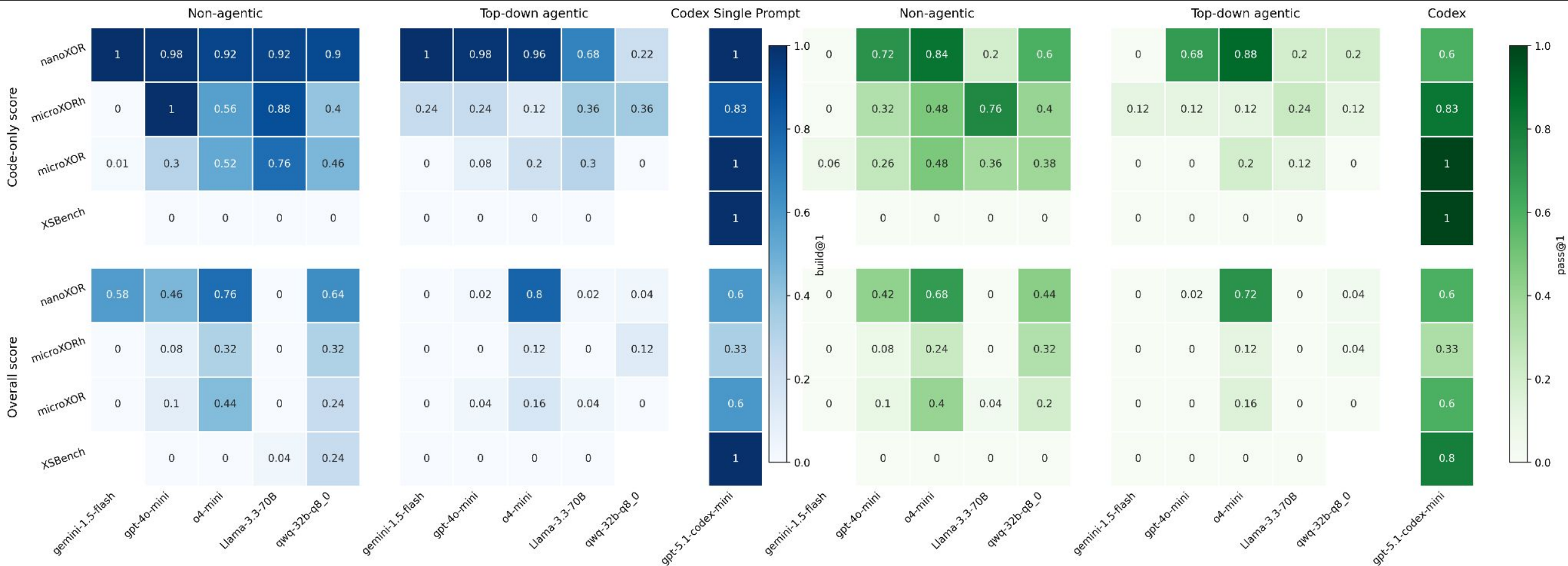
(a) build@1 for CUDA to OpenMP Offload



(b) pass@1 for CUDA to OpenMP Offload

RQ2: Can Codex Translate between Parallel API's ?

ANS2: CUDA TO OMP



RQ3: Can ParaCodex work with more complex code (NAS, Rodinia) ?
ANS3: Yes, NAS improvement in 4/4 of the Kernels

NAS Parallel Benchmarks (NPB)

- Developed by NASA to model computational fluid dynamics (CFD) workloads.
- Focus on communication patterns, and scalability.
- Kernels are compact, synthetic, representing algorithmic motifs (CG, FT, EP, MG).

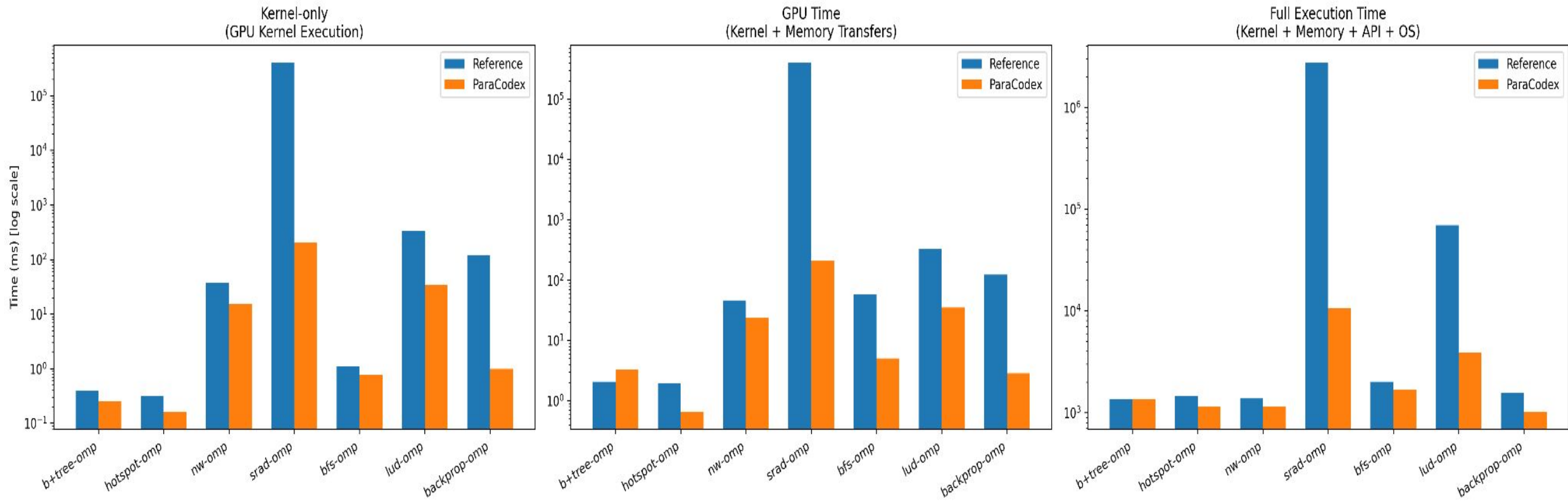
Rodinia Benchmark Suite

- Designed to represent heterogeneous workloads (CPU, GPU, accelerators).
- Targets data-parallel applications such as medical imaging, machine learning, graph algorithms, physics simulations.
- Focus on real application behavior, memory patterns, and GPU architectural stress.

*complex - Rodinia provides full real-world heterogeneous workloads, and NAS offers compact, standardized pseudo-applications designed to benchmark parallel scalability and algorithmic structure.

Rodinia Results

Performance Comparison: Reference vs ParaCortex



Benchmark	Kernel Reference (ms)	Kernel ParaCodex (ms)	Kernel Faster?	GPU Reference (ms)	GPU ParaCodex (ms)	GPU Faster?	Full Reference (ms)	Full ParaCodex (ms)	Full Faster?
b+tree-omp	0.39	0.25	1	2.06	3.28	0	1349.05	1340.27	1
hotspot-omp	0.31	0.16	1	1.95	0.66	1	1451.85	1146.99	1
nw-omp	37.83	15.19	1	45.94	23.52	1	1370.41	1139.66	1
srad-omp	403,112.75	204.32	1	403,116.81	208.24	1	2,755,297.59	10,712.70	1
bfs-omp	1.09	0.78	1	58.96	5.00	1	2013.15	1695.12	1
lud-omp	331.48	34.17	1	332.15	34.85	1	69,804.71	3886.49	1
backprop-omp	120.86	0.99	1	122.78	2.86	1	1576.15	1022.08	1

Analysis - CG

```
# Loop Analysis for CG Serial
```

```
## Loop Nesting Structure
```

```
- Main timed iteration (`data/src/cg-omp/cg.c:226`) Type E
  |— Norm dot-product (`data/src/cg-omp/cg.c:231`) Type F
  |— Vector scale (`data/src/cg-omp/cg.c:243`) Type A
  |— `conj_grad` body (dominant work)
    |— `cgit` loop (`data/src/cg-omp/cg.c:341`) Type E
      |— SpMV row loop (`data/src/cg-omp/cg.c:346`) Type B
        |— Column visit (`data/src/cg-omp/cg.c:350`) Type B
```

Analysis - EP

EP Loop Classification

Summary Table

Function	Type	Priority	Parent	Iterations	Dependencies	Issues
----- ----- ----- ----- ----- ----- -----						
`main` (`ep.c:134`)	A	SECONDARY	none	`NQ` (10)	none	<10K iterations
`main` (`ep.c:146`)	E	SECONDARY	none	`MK + 1` (=17)	recurrence on `t1`	serial RNG warm-up
`main` (`ep.c:157`)	A	CRITICAL	none	`numblks = ceil(NN/blksize)`	reductions on `sx`/`sy`	aggregated hist counts RNG in timed loop; variable block size
`main` (`ep.c:165`)	A	IMPORTANT	`ep.c:157`	`blksize` (≤ 2048)	none	repeated zeroing of `qq`
`main` (`ep.c:171`)	A	CRITICAL	`ep.c:157`	`blksize`	stage-dependent RNG + reductions	RNG inside timer; scalar reductions
`main` (`ep.c:176`)	E	IMPORTANT	`ep.c:171`	≤ 100 (data-dependent)	sequential `kk`/`t1` updates	recurrence (non-parallelizable)
`main` (`ep.c:217`)	A	CRITICAL	`ep.c:171`	`2*NK` (131072)	none	high-bandwidth RNG
`main` (`ep.c:234`)	D	CRITICAL	`ep.c:171`	`NK` (65536)	reduction on `tmp_sx/tmp_sy`	indirect writes to `qq` atomic histogram updates; reductions
`main` (`ep.c:255`)	A	IMPORTANT	none	`NQ` (10) $\times$ `blksize`	reductions on `q`/`gc`	scalar reduction (gc) needs atomic
`main` (`ep.c:316`)	A	SECONDARY	none	`NQ` (10)	none	output-only I/O

Data Details

- **Dominant compute loop:** `for (blk=0; blk < numblks; ++blk)` (`ep.c:157`)` encompasses per-block RNG, histogram, and accumulation under the timer.
- **Arrays swapped between functions?:** NO (all work is contained in `main``).
- **Scratch arrays?:** YES (`xx`` and `qq`` are per-block scratch arrays, `xx`` is deallocated via `malloc``).
- **Mid-computation sync?:** NO explicit synchronization; control flow stays within `main``.

Data Plan - EP

```
# Data Management Plan
```

```
## Arrays Inventory
```

```
List ALL arrays used in timed region:
```

Array Name	Size	Type	Init	Access
----- ----- ----- ----- -----				
`xx`	`2 * NK * blksize`	doubles (~2,147,483,648 bytes @ MK=16, blksize=2048)	working	device-generated per block R/W (device only)
`qq`	`NQ * blksize`	doubles (~163,840 bytes)	scratch	zeroed on device before each block R/W (device only)
`q`	`NQ`	doubles (80 bytes)	working	host-zeroed then mirrored on device R/W (device ↔ host)
`sx`, `sy`, `gc`		scalars	working	host-zeroed; reduced on device R/W (device ↔ host)

```
**Types:** working (main data), scratch (temp), const (read-only), index (maps)
```

```
## Functions in Timed Region
```

Function	Arrays Accessed	Frequency	Must Run On
----- ----- ----- -----			
`randlc_ep`	warm-up loop (lines 146-149)	none (only RNG constants `r23/t23/r46/t46`)	MK+1 (~17) host (serial dependency)
Block compute	(`for blk` + `for k` body)	`xx`, `qq`, `sx`, `sy`, `gc`	numblks × blksize iterations (dominant work) device (parallel RNG + histogram)
Final reduction	(`for i=0; i<NQ`, `for k=0; k<blksize`)	`qq`, `q`, `gc`	once after block loop device (reduce per-bucket counts)

Data Plan – (FT)

****Host→Device Transfers:****

- When: after host `compute_initial_conditions` and `fft_init` (pre-iter setup)
- Arrays: `host_u1_real/host_u1_imag` → `d_u1_real/d_u1_imag` (~128.6 MB total); `u_real/u_imag` → `d_u_real/d_u_imag` (~0.004 MB)
- Total H→D: ~128.6 MB (+ negligible twiddle table copy if needed)

****Device→Host Transfers:****

- When: checksum reduction returns scalars each iter (implicit); no bulk copies
- Arrays: none (only scalar results)
- Total D→H: ~0 MB

****Transfers During Iterations:** NO**

- All working/scratch data persist on device; only scalar checksum outputs return each iter

Critical Checks (for chosen strategy)

****Strategy C:****

- [x] ALL functions in iteration loop use `is_device_ptr`
- [x] Scratch arrays allocated on device (not host)
- [x] No `map()` clauses (only `is_device_ptr`)

Optimization Plan – (FT)

[] 2. Kernel Launch Overhead

- Kernel instances: 113
- Expected: $\sim \text{niter} * (\text{fft stages} + \text{evolve} + \text{checksum}) \approx 100$, aligned
- If instances $\gg$ N: Helper functions called in loop
- Root cause: Not dominant (launch API 0.3% time)
- Fix: None
- Expected gain: N/A

[] 3. Memory Transfer Bottleneck

- Transfer time: $\sim 3\%$ of total GPU time (H2D only)
- If $>50\%$ AND ratio $<2x$: Transfers correct but dominant
- Fix: Keep data strategy as-is
- Expected gain: N/A

[x] 4. Hot Kernel Performance

- Kernel: `cffts{1,2,3}_neg/pos` take $\sim 86\%$ GPU time, $\sim 48\text{--}65\text{ms}$ avg
- Root cause: Intensive indexing math inside device loops; redundant stride calculations per element
- Fix: Cache strides/bases per tile and mark device pointers `restrict` to aid compiler (ACTION 4B)
- Expected gain: $\sim 5\text{--}10\%$ kernel speed from reduced integer ops/register pressure

Optimization Plan – (MG)

```
## Strategy (priority)
1. Fuse `resid` temp + update into one `target teams loop` with register temps and `__restrict__` pointers to cut kernel count and global traffic.
2. Fuse `psinv` neighbor sums + smoothing into one kernel, removing device allocations and extra synchronization while preserving existing data mapping.

## Micro-opts
[x] const, restrict, firstprivate, cache locals

## Target
- Runtime:  $\leq 0.45s$  (Class B)
- Kernels: ~1700-1800 launches for 20 iterations
- Memory: D2H <25% of total time
```

Performance Measurement

```
# Nsight Systems Performance Metrics
```

```
## Three Measurement Types
```

```
### 1 Kernel-only
```

```
[5/7] cuda_gpu_kern_sum
```

Time (ns)	Name
...	nvkernel_conj_grad
...	nvkernel_other
...	nvkernel_small

```
**Pure GPU compute time** - Sum of all GPU kernel execution times
```

```
### 2 GPU Time
```

```
Kernel-only + Memory Transfers
```

```
[6/7] cuda_gpu_mem_time_sum
```

Time (ns)	Operation
...	[CUDA memcpy HtoD]
...	[CUDA memcpy DtoH]
...	[CUDA memset]

Performance Measurement

```
### 3 Full Execution Time
```

```
```
```

```
Kernel + Memory + CUDA API + OS Runtime
```

```
[4/7] cuda_api_sum [3/7] osrt_sum
```

| ... | sync   | ... | poll  |
|-----|--------|-----|-------|
| ... | launch | ... | ioctl |

```
Total runtime - Everything (compute + memory + API overhead + OS calls)
```

```

```

```
Summary
```

| Metric                 | Components       | Use Case                 |
|------------------------|------------------|--------------------------|
| -----                  | -----            | -----                    |
| <b>**Kernel-only**</b> | GPU kernels only | Pure compute performance |
| <b>**GPU Time**</b>    | Kernels + Memory | GPU-side                 |
| <b>**Full Time**</b>   | All components   | Full Application         |



# NAS Results [CG, EP, FT, MG] (serial -> OMP)

Performance Comparison: Reference vs ParaCodex vs codex

